Sample Design of Socket Programming Project

Following are the instructions in the project assignment. This manual addresses the points highlighted in red below.

Your design document must cover:

- Server data structures used to map topics to directories.
- Client-server message formats, for Part I and for every case of Part II (Case I, Case II, and Case II – Enhanced).
- Fault injection mechanism planned to be used in your work.
- Details of session management protocol: sequence and types of communicated messages, message formats used by the client and server, what information is stored by the server/clients.
- Performance enhancements planned in the implementations
- Any extra or additional properties/features of your implementation beyond the minimum requirements (for example: logging, configurability, graceful shutdown, extensibility to new topics) that you plan to include, and why.
- How your code will look: the overall module/file structure, the key functions or classes you plan to write, and how the single-threaded, multi-threaded, and fault-tolerant logic fit together.
- A short write-up of your design thought process for the project - the alternatives you considered for each case and why you chose your final approach.

Server data structures used to map topics to directories

Each entry can be represented as:

```
struct TopicEntry {
   char topic[MAX_TOPIC_LEN];
   char directory[MAX_PATH_LEN];
};
```

The complete mapping can be represented using an array or linked list of type struct TopicEntry. Your server implementation should use a configuration file to populate this table at server startup. This avoids hard-coding directory paths into the server code. Appropriate error checking should be done to ensure that the directories for each topic are present and accessible in the server file system.

Client-server communication for Part 1 (single threaded)

Application layer messages will have the common format

Header	Payload

At minimum, the header will have message type, transaction ID to match the response, and payload length. Please feel free to add more fields if a given message type requires.

Message Types:

Message Type	Communication	Remark
GET <topic >	From client to server	request from client to retrieve TOPIC related information from the server
OK <topic directory name and list of files>	From server to client	Server informs client the existence of requested files in the server file system. This also specifies the sequence in which the files will be served to the client.
ERR <error code, error information>	From server to client	Server informs client about error in providing files. Error code represents the reason.
START <topic>	From client to server	Client requests transfer of all the files of the topic
FILE_HDR <topic, file name or other file identifier, file attributes>	From server to client	This message is sent before starting each file transfer. File attributes can be file type, size, permissions, etc.
DATA <additional data header:e.g., topic, file name, offset, size> <file chunk of the specified size>	From server to client	These messages carry the file data.
ACK <header with transaction ID> <topic, file, chunk offset etc. for DATA message>	From client to server	Acknowledges successful receipt of FILE_HDR or DATA chunk to the server.
CLOSE	From server to client	Used when the topic transfer completes.

Identify and use additional message types if required. Provide appropriate justification.

Part 1 Design with Single threaded and Multi-threaded Server

The single-threaded server has one main execution thread. The server does not accept and process another client until the current transfer has completed. Consequently, if one client requests a large topic, other clients must wait. This implementation provides the baseline for the performance comparison.

The multi-threaded server uses one worker thread per connected client. The main thread performs only connection acceptance and worker-thread creation. Each worker thread independently interacts with client and serves requested topic(s). The following pseudo code specifies how the topic transfer from server to client is carried out using the aforementioned messages:

Server side	**Client side**
message = receive_message(client_socket) IF message.type != GET: Send ERR(invalid_request) Close client_socket RETURN	Send GET(topic)
topic = message.topic IF topic not in topic_to_directory: Send ERR(unknown_topic, topic) Close client_socket RETURN directory = topic_to_directory[topic] files = enumerate regular files in directory Send OK(directory_name, file_list)	message = receive_message() IF message.type == ERR: Display error Close socket EXIT IF message.type != OK: Handle protocol error Close socket EXIT
message = receive_message(client_socket) IF message.type != START OR topic mismatch: Send ERR(invalid_start, topic) Close client_socket RETURN	directory_name = message.directory_name files = message.file_list Create local directory for topic Send START(topic)
FOR each file in files: Open file, Determine file attributes Send FILE_HDR(topic, file attributes) offset = 0 WHILE offset < file_size: Read next chunk from file chunk_size = number of bytes read Send DATA(topic, file_name, offset, chunk_size, chunk) offset = offset + chunk_size Close file Send CLOSE Close client_socket	WHILE TRUE: message = receive_message() IF message.type == CLOSE: BREAK ELSE IF message.type == ERR: Display error BREAK ELSE IF message.type == FILE_HDR: file_name = message.file_name file_size = message.file_size Open local file for writing offset = 0 WHILE offset < file_size: message = receive_message() IF message.type != DATA: Handle protocol error, remove local file, Close socket, EXIT Validate message.topic, file name/index, offset, chunk size etc Write chunk to local file offset = offset + message.chunk_size Close local file Display "Transfer completed" Close socket

Fault injection mechanism for Part 2

Please note that this is only one way of fault injection. There are other correct ways as well. As specified in the project description, the server accepts failure probability as a command line argument. For example:

./server <port> 0.01

represents a 1% failure probability. A random decision is made periodically during an active transfer. The failure decision using a random number generator is made after a successful data chunk has been transmitted and before the next chunk is sent. For example:

```
while (offset < file_size) {
    send next chunk;
    update offset;
    if (random_failure_event(failure_probability)) {
        close(connection);
        return;
    }
}
```

Part II Case 1: No Session Management

In Case 1, client or server does not maintain transfer state. The client detects the disconnection and establishes a new TCP connection. At implementation level, there are only minor changes required compared to Part - 1:

- In the server program: add probability based connection closure.
- In the client program:
  - add detection of connection closure
  - Remove the topic directory
  - re-establish a fresh connection with the server and request the topic again

The server treats every new connection as a new topic transfer.

Following is the simplified pseudo-code of server and client for this case. For the performance comparison, some record keeping is additionally required at client and server.

Server side	**Client side**
	transfer_completed = FALSE WHILE NOT transfer_completed: CONNECT TO SERVER
message = receive_message(client_socket) IF message.type != GET: Send ERR(invalid_request) Close client_socket RETURN	Send GET(topic)
topic = message.topic IF topic not in topic_to_directory: Send ERR(unknown_topic, topic) Close client_socket RETURN directory = topic_to_directory[topic] files = enumerate regular files in directory Send OK(directory_name, file_list)	message = receive_message() IF message.type == ERR: Close socket EXIT IF message.type != OK: Handle protocol error Close socket EXIT

```
message = receive_message(client_socket)
IF message.type != START OR topic mismatch:
    Send ERR(invalid_start, topic)
    Close client_socket
    RETURN
FOR each file in files:
    Open file, Determine file attributes
    Send FILE_HDR(topic, file attributes)
    offset = 0
    WHILE offset < file_size:
        Read next chunk from file
        chunk_size = number of bytes read
        Send DATA(topic, file_name, offset,
                chunk_size, chunk)
        offset = offset + chunk_size
        IF random_failure_event(probability):
                Close file
                Close client_socket
                RETURN
    Close file
Send CLOSE
Close client_socket
```

```
    directory_name = message.directory_name
    files = message.file_list
    Create local directory for topic
    Send START(topic)
    attempt_failed = FALSE
    WHILE TRUE:
        message = receive_message()
        IF connection closed:
            attempt_failed = TRUE
            BREAK
        IF message.type == CLOSE:
            transfer_completed = TRUE
            BREAK
        ELSE IF message.type == ERR:
            Display error
            BREAK
        ELSE IF message.type == FILE_HDR:
            file_name = message.file_name
            file_size = message.file_size
            offset = message.offset (it will be zero)
            Open local file for writing
            WHILE offset < file_size:
                message = receive_message()
                IF connection closed:
                    Close local file
                    attempt_failed = TRUE
                    BREAK
                IF message.type != DATA:
                    Handle protocol error,
                     remove local file, Close socket,  EXIT
                Validate message.topic, file name/index,
                offset, chunk size etc
                Write chunk to local file
                offset = offset + message.chunk_size
            Close local file
            IF attempt_failed:
                BREAK
    Display "Transfer completed"
    Close socket
```

In this pseudo-code client handles errors that are generated during chunk transfer. However, if the server can generate an error while communicating other messages, e.g., OK or FILE_HDR, it needs to be handled accordingly.

Part II Case 2: Session Management

Each transfer is identified using a unique session_id which can be a 32-bit or 64-bit unique identifier. The session state may contains:

```
struct SessionState {
   uint64_t session_id;
   char topic[MAX_TOPIC_LEN];
   int file_index or char filename[MAX_FILENAME_LEN];
   uint64_t file_offset;
   time_t last_update;
   other information for record keeping
};
```

The server maintains active session state in a session table. The client maintains the corresponding checkpoint locally. This may be considered analogous to Cookies in HTTP. The session ID will be shared in all the messages related to a topic transfer : GET (on resuming session), OK, START, FILE_HDR, DATA, ACK, CLOSE.

- When a client initiates a new topic transfer
  - The client sends a GET message. This is same as earlier.
  - The server creates a new SessionState and sends session_id in its OK message.
  - The client saves and sends this session ID in all its communications to the server (START, ACK)
  - Server uses the session ID and information shared in the ACK message to update the Session state.
- On reconnecting after failure
  - Client initiates new connection and sends GET <TOPIC> message with existing sessionID
  - On receiving a GET message with session ID
    - Server looks up and retrieves the session state
    - responds with OK message with list of remaining files only.
  - Client Sends START message which contains the session ID
  - Server carries out file transfer loop only for remaining files or the part of a file (FILE_HDR message will have file size specifying remaining transfer only for the file).

Following is the simplified pseudo-code of server and client for this case.

Server side	**Client side**
	transfer_completed = FALSE WHILE NOT transfer_completed: CONNECT TO SERVER
message = receive_message(client_socket) IF message.type != GET: Send ERR(invalid_request) Close client_socket RETURN	Send GET(topic with/without sessionID)
topic = message.topic IF topic not in topic_to_directory: Send ERR(unknown_topic, topic) Close client_socket RETURN	message = receive_message() IF message.type == ERR: Display error Close socket EXIT

```
directory = topic_to_directory[topic]
files = enumerate regular files in directory
if GET message has SessionID, retrieve session state
Send OK[new/existing sessionID]
(directory_name, file_list)
```

```
    IF message.type != OK:
        Handle protocol error
        Close socket
        EXIT
```

```
message = receive_message(client_socket)
IF message.type != START OR topic mismatch:
    Send ERR(invalid_start, topic)
    Close client_socket
    RETURN
```

```
    directory_name = message.directory_name
    files = message.file_list
    Create local directory for topic
    Send START(topic)
```

```
FOR each file in files or remaining files in a restored session:
    Open file, Determine file attributes
    Send FILE_HDR(topic, file attributes)
    offset = 0 or restored offset from session
    WHILE offset < file_size:
        Read next chunk from file
        chunk_size = number of bytes read
        Send DATA(topic, file_name, offset,
                chunk_size, chunk)
        offset = offset + chunk_size

        IF random_failure_event(probability):
                Close file
                Close client_socket
                RETURN
    Close file
Send CLOSE
Close client_socket
```

```
    attempt_failed = FALSE
    WHILE TRUE:
        message = receive_message()
        IF connection closed:
            attempt_failed = TRUE
            BREAK
        IF message.type == CLOSE:
            transfer_completed = TRUE
            BREAK
        ELSE IF message.type == ERR:
            Display error
            BREAK
        ELSE IF message.type == FILE_HDR:
            file_name = message.file_name
            file_size = message.file_size
            offset = message.offset
            Open local file for writing and seek to offset
            WHILE offset < file_size:
                message = receive_message()
                IF connection closed:
                    Close local file
                    attempt_failed = TRUE
                    BREAK
                IF message.type != DATA:
                    Handle protocol error,
                     remove local file, Close socket,  EXIT
                Validate message.topic,
                file name or index, offset, chunk size etc
                Write chunk to local file
                offset = offset + message.chunk_size
            Close local file
            IF attempt_failed:
                BREAK
    Display "Transfer completed"
    Close socket
```

Note: We have seen few of your submissions, and some designs are really good – even better than this simplistic minimal model design.

Evaluation scheme for Design Document:

- Server data structures used to map topics to directories. **[2 marks]**
- Client-server message formats, for Part I and for every case of Part II (Case I, Case II, and Case II – Enhanced). **[10 marks]**
- Design of Part 1 **[10 marks]**
- Fault injection mechanism planned to be used in your work. **[3 marks]**
- Design of Part 2 – Case 1 **[5 marks]**
- Details of session management protocol and design of Part 2- Case 2 **[10 marks]**
- Performance enhancements planned in the implementations **[5 marks]**
- A short write-up of your design thought process for the project - the alternatives you considered for each case and why you chose your final approach. **[5 marks]**
- **Total Marks: 50**